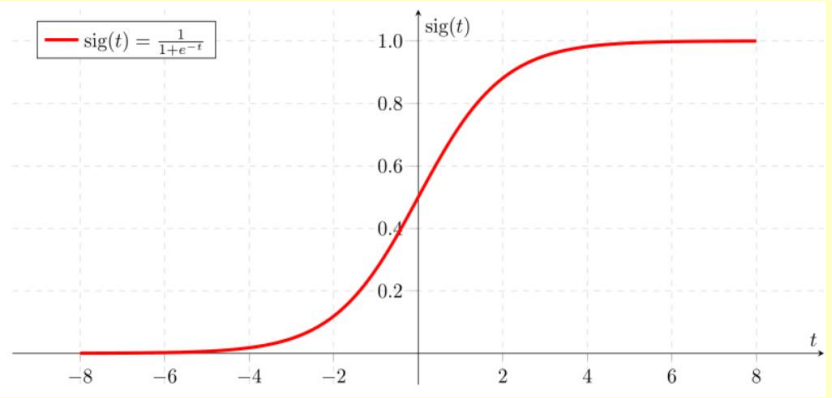
Sigmoid Function : La fonction pour la régression Logistique

la fonction sigmoid (**Sigmoid Function)** produit des valeurs comprises entre 0 et 1.



$$sigmoide(x) = \frac{1}{1+e^{-x}}$$

Sigmoid Function : La fonction pour la régression Logistique

- La régression logistique est utilisée pour trouver la probabilité d'un événement. On veut déterminer le succès ou l'échec d'un événement.
- Largement utilisé pour les problèmes de classification
- Ne nécessite pas de relation linéaire entre les variables dépendantes et indépendantes.
- La régression logistique permettra de répondre à des problèmes comme :

  Est-ce que le client est solvable pour lui accorder un crédit ou non?

  Est-ce que la tumeur diagnostiquée est bénigne ou maligne ?

  Est-ce qu'un message est spam ou non ?

Sigmoid Function : La fonction pour la régression Logistique

- La fonction hypothèse:

$$S(X^{(i)}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + ... + \theta_n x_n$$

- $X^{(i)}$: une observation (du *Training Set* ou du *Test Set*), cette variable est un vecteur contenant: $[x_1, x_2, x_3, ...x_n]$
- x_i : est une variable prédictive (feature) qui servira dans le calcul du modèle prédictif
- θ_i : est un poids/paramètre de la fonction hypothèse. Ce sont ces θ_i qu'on cherche à calculer pour obtenir notre fonction de prédiction.
- θ_0 : est une constante nommée le **bias (biais)**

63

Sigmoid Function : La fonction pour la régression Logistique

- On peut réécrire $\theta_0 = \theta_0 * x_0$: avec $x_0=1$, Cela nous permet de réécrire notre fonction S(X) de façon plus compacte comme suit : $S(X) = \sum_{i=0}^{n+1} \theta_i * x_i$

On appelle cette fonction hypothèse : **la fonction score.** L'idée est de trouver des coefficients: $[\theta_0, \theta_1, \theta_2, \theta_3, ...\theta_n]$

de sorte que :

- $S(X^{(i)}) > 0$: quand la classe (étiquette) vaut 1
- $S(X^{(i)}) < 0$: quand la classe (étiquette) vaut 0

$$\Pr obabilité(X \in Classe"1") = Sigmoide(\theta.X) = \frac{1}{1+\exp(-S(X))} = \frac{1}{1+\exp(-\theta.X)}$$

Le résultat obtenu par la fonction **sigmoid (θ.X)** est interprété comme la **probabilité que l'observation X soit d'un label (étiquette) 1.**

64

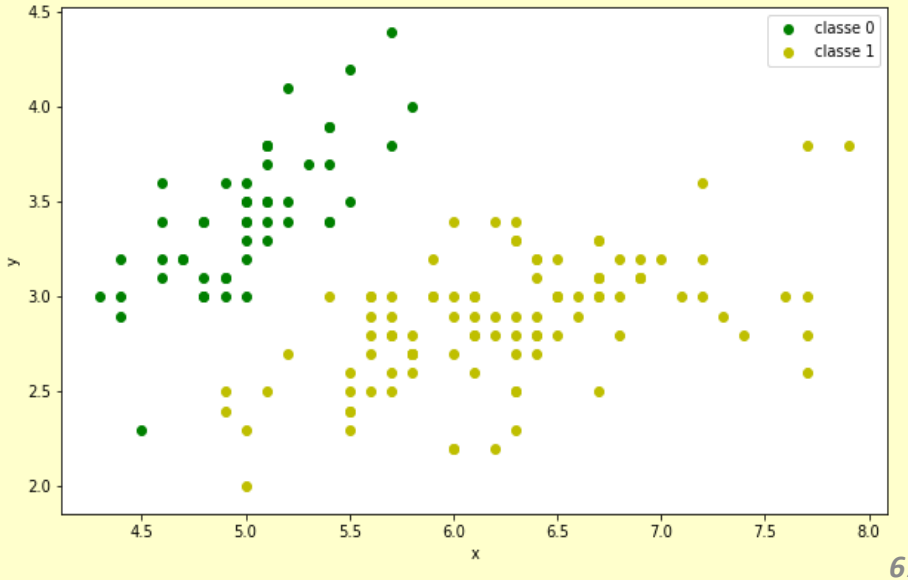
la régression Logistique implémenté dans sklearn

```
import numpy as np  # Chargement de numpy
import matplotlib.pyplot as plt # import de Matplotlib
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
iris = datasets.load_iris()
X = iris.data[:, :2] # avoir un problème de classification binaire.
# re-etiquettage des fleurs pour se retrouver avec deux classes au lieu de trois
y = []
for e in iris.target:
    if e==0:        y+=[0]
    else:        y+=[1]
y=np.array(y)
```


la régression Logistique implémenté dans sklearn

```
plt.figure(figsize=(10,6)) # Taille de la figure
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='g')
# En Vert les fleurs ayant l'étiquette 0
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='y')
# en Jaune les fleurs ayant l'étiquette 1
plt.legend(labels = ("classe 0", "classe 1"));
plt.xlabel("x")
plt.ylabel("y")
```

la régression Logistique implémenté dans sklearn



la régression Logistique implémenté dans sklearn

```
model = LogisticRegression(C=1e20)#tester avec 0.01
x_train, x_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=0)
#la fonction train_test_split mélange les données avant de faire la répartition
#si on fixe random_state sur une valeur précise, on exécute sur deux machines différentes
#les le mélange sera fait de la même façon sur les deux machines et on obtient les mêmes
model.fit(x_train, y_train)                                                    test_set
score=cross_val_score(model,x_train, y_train, cv=5,scoring='accuracy')
score1=model.score(x_test, y_test)
score2=model.score(x_train, y_train)
print("score sur test-set ="+str(score1*100)+'%')
print("score sur train-set ="+str(score2*100)+'%')
theta0=model.intercept_;  theta1=model.coef_[0][0]; theta2=model.coef_[0][1]
Iries_To_Predict = [    [5.5, 2.5],    [7, 3],    [3,2],    [5,3]]
# demande de prédiction
print(model.predict(Iries_To_Predict))
```

```
score sur test-set =100.0%
score sur train-set =100.0%
[1 1 0 0]
```

la régression Logistique implémenté dans sklearn

```
score=cross_val_score(model,x_train, y_train, cv=5,scoring='accuracy')
print(score)
print(score.mean())
def fct_reg_logistic(x1,x2):
    z=(theta0)+theta1*x1+(theta2*x2)
    return(z)
print([fct_reg_logistic(e[0],e[1]) for e in Iries_To_Predict])
```

```
[1.         0.95833333 1.         1.         1.        ]
0.9916666666666668
[array([24.57979467]), array([60.629162]), array([-45.97305733]), array([-8.37780735])]
```


la régression Logistique implémenté pas à pas

La probabilité de prédiction de la classe positive est déterminée par:

$$h(\theta, x) = \frac{1}{1+e^{-\theta.x}}$$

La fonction de coût à minimiser :

$$j(\theta) = \frac{1}{m}\sum_{1}^{m}[(0-y^i)*\log(h(x^i)) - (1-y^i)*\log(1-h(x^i))]$$

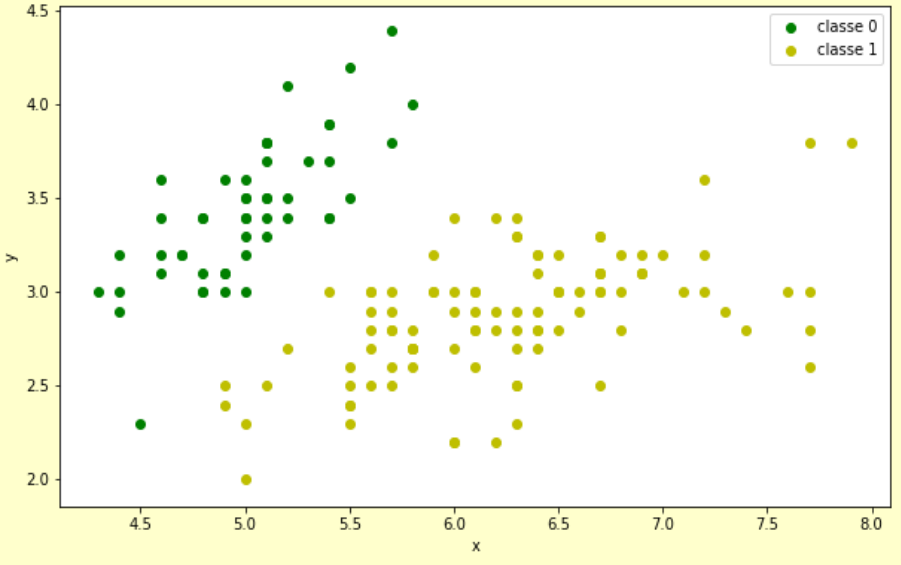
Le gradient est: $$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}(h(\theta, x^{(i)}) - y^{(i)}).x_j^{(i)}$$

la régression Logistique implémenté pas à pas

```
import  numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
iris = datasets.load_iris()
x= iris.data[:, :2] # avoir un problème de classification binaire.
y = []
for e in iris.target:
    if e==0:y+=[0]
    else:y+=[1]
y=np.array(y)
```

```
plt.figure(figsize=(10,6)) # Taille de la figure
plt.scatter(x[y == 0][:, 0], x[y == 0][:, 1], color='g', label='0')
# En Vert les fleurs ayant l'étiquette 0
plt.scatter(x[y == 1][:, 0], x[y == 1][:, 1], color='y', label='1')
# en Jaune les fleurs ayant l'étiquette 1
plt.legend(labels = ("classe 0", "classe 1"));
plt.xlabel("x")
plt.ylabel("y")
```


la régression Logistique implémenté pas à pas



la régression Logistique implémenté pas à pas

$$h(\theta, x) = \frac{1}{1+e^{-\theta.x}}$$

$$j(\theta) = \frac{1}{m}\sum_{1}^{m}[(0-y^i)*\log(h(x^i)) - (1-y^i)*\log(1-h(x^i))]$$

```
X=np.concatenate((np.ones((y.shape[0],1)),x),axis=1)
theta = np.zeros(X.shape[1])
def h(theta,x):
    return 1/(1+np.exp(-np.dot(theta.T,x)))

def Erreurs_j_theta(x,y,theta):
    m=x.shape[0]
    s=0
    for i in range(m):
        s+=y[i]*np.log(h(theta,x[i]))+(1-y[i])*np.log(1-h(theta,x[i]))
    return(-s/m)
```

73

la régression Logistique implémenté pas à pas

Le gradient est: $$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}(h(\theta, x^{(i)}) - y^{(i)}).x_j^{(i)}$$

```
def gradient_D_j_theta(x,y,theta,j):
    m=x.shape[0]
    s=0
    for i in range(m):
        s+=(h(theta,x[i])-y[i])*x[i][j]
    return(s/m)
```

74

la régression Logistique implémenté pas à pas

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}(h(\theta, x^{(i)}) - y^{(i)}).x_j^{(i)}$$

```
def gradient_descent(X, y, theta, learning_rate, n_iterations):
    # création d'un tableau de stockage pour enregistrer l'évolution des erreurs
    historique_des_erreurs = np.zeros(n_iterations)
    for i in range(0, n_iterations):
        H=np.array([h(theta,x) for x in X])
        Hy=[abs(H[u]-y[u]) for u in range(X.shape[0])]
        theta[0] = theta[0] - (learning_rate/X.shape[0]) * sum(Hy)
        for j in range(X.shape[1]):
            theta[j] = theta[j] - learning_rate*gradient_D_j_theta(X, y, theta,j)
        historique_des_erreurs[i] = Erreurs_j_theta(X,y,theta)
    return theta, historique_des_erreurs
```

75

la régression Logistique implémenté pas à pas

```
# Example de test :
n_iterations = 10000
learning_rate = 0.01
theta_final, historique_des_erreurs=gradient_descent(X, y,theta, learning_rate, n_iterations)
print(theta_final) # thetaune fois que la machine a  été entrainée
def h_avec_ajout_biais_a_x(theta,x):
    XX=np.concatenate((np.array([1]),x),axis=0)
    return 1/(1+np.exp(-np.dot(theta.T,XX)))
```

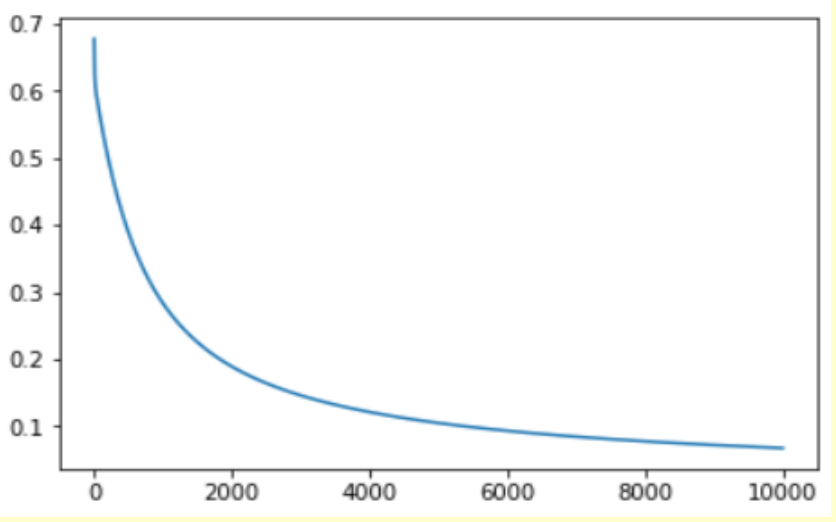
```
[-12.42617591   4.4185545   -3.63355506]
```

76

la régression Logistique implémenté pas à pas

```
#calcul du taux de réussite sur l'ensemble d'apprentissage:
cpt=0
for i in range(y.shape[0]):
    t=h_avec_ajout_biais_a_x(theta_final,x[i])
    if t<0.5:
        target=0
    else:
        target=1
    if target==y[i]:
        cpt+=1
    else:
        print('Problème au point:',x[i])
print((cpt/y.shape[0])*100)
plt.plot(range(n_iterations),historique_des_erreurs)
plt.show()
```

```
100.0
```


la régression Logistique implémenté pas à pas

l'évolution des erreurs au cours des itérations